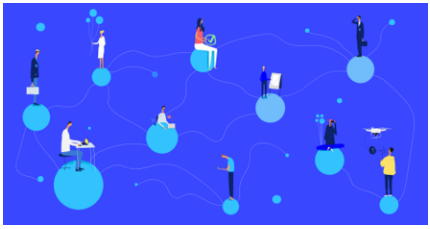
Optimization for Deep Learning (OPT4DL)

Abolfazl Hashemi, Purdue ECE

Distributed Deep Learning

Learning from Distributed Data



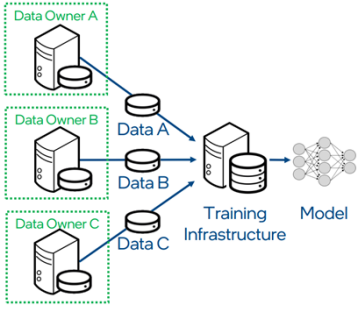
- So far, we discussed training tasks such as minimizing $f(w) = \frac{1}{n}\sum_{i=1}^{n} \ell(w, z_i)$ (or abstractly, $f(w) = \mathbb{E}_{z\sim D}[\ell(w, z)]$) assuming data is all in one place
- In many settings, data is generated and is held locally by many devices or agents. Essentially,

$$\min_w f(w) = \frac{1}{K}\sum_{j=1}^{K} f_j(w), \qquad f_j(w) = \mathbb{E}_{z\sim D_j}[\ell(w, z)] \tag{1}$$

- We call f and f_j the global and local loss respectively.

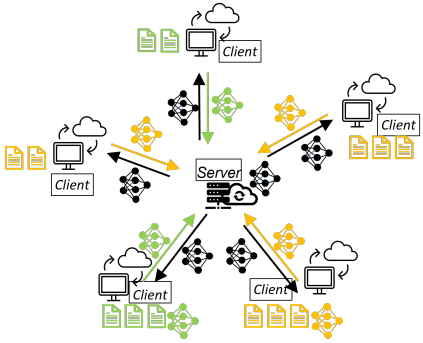
Centralized Learning

- All agents send their data to a central entity called Server
- Remark: We
  use Server, aggregator, and fusion center interchangeably
- Remark: We use agent, client,
  worker, participant, device, and node interchangeably
- Features
  - Data integrity no preserved
  - Communication
    issues if data size larger than agents' capability
  - Potential bottleneck at server if many agents
  - Fast and accurate training



Parallel Learning

- Instead of sharing
  raw data, agents share model-based representation of local data through (repeatedly) communicating $\nabla f_j(w_t)$ or similar quantities with server
- For $t = 1, \ldots, T$ Communication/training rounds
  - server sends the current global model $\bar{w}_t$ to the agents
  - each agent computes and
    sends $\nabla f_j(w_t)$ or $w_{t+1}^j = \bar{w}_t - \eta \nabla f_j(w_t)$ to the server
  - Server aggregates
    the recieved messages and update the global model



$$\bar{w}_{t+1} = \bar{w}_t - \eta \frac{1}{K} \sum_{j=1}^{K} \nabla f_j(w_t), \qquad \bar{w}_{t+1} = \frac{1}{K} \sum_{j=1}^{K} w_{t+1}^j \tag{2}$$

Parallel Learning (Cont'd)



- Features
  - Improved level of preserving data integrity
  - Taking advantage of local computation
  - communication issues if model size larger than agents' capability
  - Potential bottleneck at server if many agents
  - Delay duo to stragglers in synchronous systems
- We think of the averaging operator

$$\bar{w}_{t+1} = \bar{w}_t - \eta \frac{1}{K} \sum_{j=1}^{K} \nabla f_j(w_t), \qquad \bar{w}_{t+1} = \frac{1}{K} \sum_{j=1}^{K} w_{t+1}^j \tag{3}$$

  as an aggregation mechanism. Essentially maximizing Utilitarian Welfare.
- Can define other mechanisms based on other notions, e.g., egalitarian welfare (minimizing the worst loss).



- Can be thought of as approximate Parallel Learning to reduce communication and computation by trading off accuracy and precision
- Periodic Communication: Instead of communicating local gradients or local model updates after 1 local step, do so after $E > 1$ local steps

$$w^j_{t,\tau+1} = w^j_{t,\tau} - \eta \nabla f_j(w^j_{t,\tau}), \quad \tau = 1, \ldots, E, \quad w^j_{t,1} = \bar{w}_t, \quad w^j_{t,E+1} = w^j_{t+1} \tag{4}$$

- Can be thought of as approximate solving minimizing the local loss when initializing at $\bar{w}_t$:

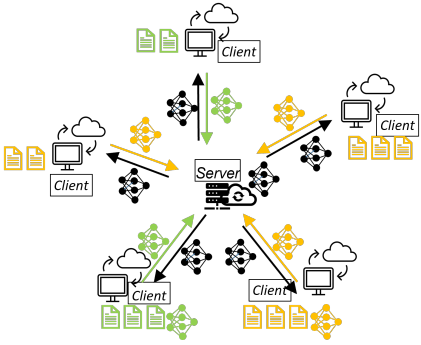
$$w^j_{t,E+1} = w^j_{t+1} \approx \text{argmin } f_j(w) = \mathbb{E}_{z \sim D_j}[\ell(w, z)] \tag{5}$$

- Partial Participation: Each communication round, only a subset S_t of $r < N$ agents participate

$$\bar{w}_{t+1} = \frac{1}{r} \sum_{j \in S_t} w^j_{t+1} \tag{6}$$

Hardware Heterogeneity

- Agents may have
  different communication and computation capabilities
- Computation: agent i could
  be able to do more local steps than agent j, $E_i > E_j$
- Communication 1: agent i could
  be able to participate more frequently than agent j
- Communication 2: agent i could be able to send
  higher precision messages (more bits) than agent j
- Could lead to fairness issues
  as the typical method could favor more capable clients





- Each agent holds different loss as a result of different data, $D_i \neq D_j$
- Integration of partial participation and periodic communication leads to convergence issues.
- Let us rewrite our simple aggregation

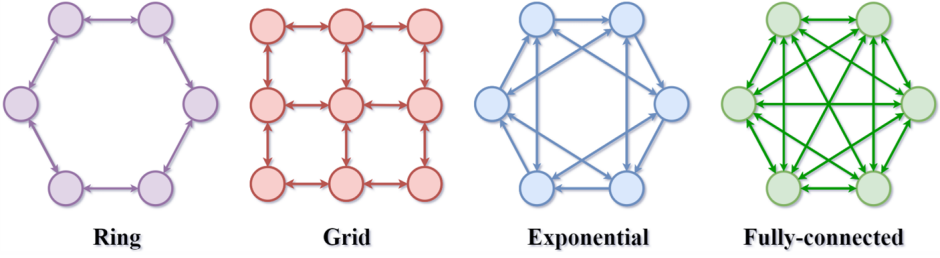
$$\begin{aligned} \bar{w}_{t+1} &= \frac{1}{r}\sum_{j\in S_t} w_{t+1}^j = \frac{1}{r}\sum_{j\in S_t} \bar{w}_t - \eta \sum_{\tau=1}^{E} \nabla f_j(w_{t,\tau}^j) \\ &= \bar{w}_t - \eta \frac{1}{r}\sum_{j\in S_t}\sum_{\tau=1}^{E} \nabla f_j(w_{t,\tau}^j) := \bar{w}_t - \eta \frac{1}{r}\sum_{j\in S_t} g_{t,E}^j \end{aligned} \tag{7}$$

  Resembling mini-batch SGD.

- One way to deal with these issues then is to use aggregations mimicking momentum to reduce the variance stemming from statistical heterogeneity.

Decentralized Learning

- No server, only peer-to-peer (p2p) communications, effectively each agent serving as its own local aggregator
- Mathematically, communication governed by a graph
- Lower graph degree implies lower communication bottleneck and lower delay
- Lower connectivity implies slower propagation of information and hence a slower convergence
- Synchronous communication could still cause delay issues



Decentralized Learning (Cont'd)



- Let N_j be the set of neighbors of agent i and note $i \in N_i$
- Each agent computes and sends the aggregation of the messages recieved from its neighbors:

$$w_{t+1}^i = \Big(\frac{1}{|N_i|}\sum_{j\in N_i} w_t^j\Big) - \eta\nabla f_i(w_t^i), \qquad w_{t+1}^i = \frac{1}{|N_i|}\sum_{j\in N_i}\Big(w_t^j - \eta\nabla f_i(w_t^j)\Big) \tag{8}$$

- Remark: A fully connected graph with no server node is equivalent to a star graph with a server node. Thus, parallel learning is an example of Decentralized Learning

- Remark: Federated Learning is also an example of Decentralized Learning with a sequence of (disconnected) time-varying graphs